

Python OOP

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 04, Self-study

22 September 2026

1. Where This Fits
2. Learning Outcomes
3. Attributes and Methods
4. Inheritance
5. Special Methods and the Dataset Pattern
6. A Model as a Class
7. Olympiad Practice
8. Check Yourself
9. Limitations and What Comes Next
10. Summary
11. References

- ▶ **In the lecture (Day 01):** a **class** is a blueprint, an **object** is one thing built from it. `__init__` sets up the attributes, `self` means “this object”, and `object.method()` does an action. You wrote `class Player` and a `Student` class in Lab 1.
- ▶ **This deck adds:** printing objects nicely, methods that return versus methods that change, **inheritance** (a class built on another), the **special methods** `__len__` and `__getitem__`, and a complete **model class** with `fit` and `predict`.
- ▶ **How to use it:** nobody is presenting. Read one frame, type its code into a Colab cell, run it, compare with the output printed on the frame. Answer the three questions at the end before looking at the answers.

Where the lecture's version breaks

Day 01's Player works alone. In Week 3 your class must plug into someone else's code: PyTorch will call `len(dataset)` and `dataset[i]` on an object *you* wrote, and expects your network to be built on its `nn.Module`. If your class does not follow those shapes, the library raises an error before training starts. Every scikit-learn model in Week 2 is an object with `fit` and `predict`; you will use dozens of them.

After this deck you can

Read any class in a library notebook and say what its attributes hold and what each method does; build on a library class with inheritance; write a dataset class and a model class in the shape the libraries expect.

By the end of this deck, you will be able to:

- ▶ **Explain** the difference between an attribute and a method, and between a method that returns and one that changes the object.
- ▶ **Write** `__str__` so that `print(obj)` shows something useful.
- ▶ **Build** a child class with inheritance, call the parent's `__init__` with `super()`, and override a method.
- ▶ **Implement** `__len__` and `__getitem__` so that `len()`, indexing and `for` work on your object.
- ▶ **Write** a model class with `fit` and `predict` that stores what it learned in attributes.
- ▶ **Recognise** these shapes in scikit-learn (Week 2) and PyTorch (Week 3) code.

Python OOP: **Attributes and Methods**

```
class Player:
    def __init__(self, name):
        self.name = name          # attribute
        self.score = 0           # attribute

    def add_points(self, n):      # method
        self.score = self.score + n

    def level(self):              # method
        return self.score // 10

p = Player("Sara")
p.add_points(23)
print(p.name, p.score, p.level())
# Sara 23 2
```

- ▶ **Attributes** are the object's data: `self.name`, `self.score`. They are created inside `__init__` and live as long as the object.
- ▶ **Methods** are functions defined inside the class. Their first parameter is always `self`; Python fills it in when you write `p.add_points(23)`.
- ▶ `p.score` reads an attribute (no brackets). `p.level()` calls a method (brackets, even with no arguments); $23 // 10 = 2$.

Changes the object, returns nothing

```
def add_points(self, n):  
    self.score = self.score + n
```

```
result = p.add_points(5)  
print(result)          # None  
print(p.score)         # 28
```

Returns a value, changes nothing

```
def level(self):  
    return self.score // 10
```

```
lvl = p.level()  
print(lvl)              # 2  
print(p.score)          # 28  
                        unchanged
```

The mistake this frame prevents

Writing `p = p.add_points(5)` replaces the player with `None`, because `add_points` returns nothing. The next `p.score` then fails with `AttributeError: 'NoneType' object has no attribute 'score'`. Rule: a method either changes `self` or returns an answer. When in doubt, read its last line.

```
print(p)                # <__main__.Player object at 0x7f3c...>

class Player:
    def __init__(self, name):
        self.name = name
        self.score = 0

    def __str__(self):      # called by print() and str()
        return f"Player {self.name}: {self.score} points"

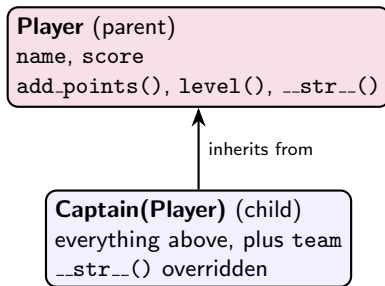
p = Player("Sara")
print(p)                 # Player Sara: 0 points
```

- ▶ Names with two underscores on each side are **special methods**: Python calls them at fixed moments. `__init__` runs when an object is built; `__str__` when it is printed. You never call them yourself.
- ▶ The next sections use two more: `__len__` and `__getitem__`.

Python OOP: **Inheritance**

Definition

A **child class** inherits everything from a **parent class**: all attributes and all methods. The child then adds new attributes or methods, or **overrides** (replaces) a parent method with its own version.



Why: a captain *is a* player with one extra field. Without inheritance you would copy the whole **Player** class and keep two copies in sync (Day 01's "why functions" argument, one level up).

```
class Captain(Player):           # the brackets name the parent
    def __init__(self, name, team):
        super().__init__(name)   # Player's __init__: name, score
        self.team = team         # then the new attribute

    def __str__(self):           # override: replaces Player's
        return f"Captain {self.name} ({self.team}): {self.score} points"

c = Captain("Omar", "red")
c.add_points(12)                # inherited, not written again
print(c)                        # Captain Omar (red): 12 points
print(c.level())                # 1    inherited too
```

- ▶ super() means “the parent class”: super().__init__(name) does the parent's setup once. A method defined in the child wins over the parent's version; everything else is inherited unchanged.

```
import torch.nn as nn

class Net(nn.Module):                                # child of PyTorch's Module
    def __init__(self):                               # the parent's setup, first
        super().__init__()                          # an attribute: a layer object
        self.layer = nn.Linear(4, 3)

    def forward(self, x):                             # the method PyTorch will call
        return self.layer(x)
```

- ▶ The same three lines as Captain: brackets name the parent, `super().__init__()` first, then your attributes, then your methods.
- ▶ `nn.Module` gives `Net` dozens of methods for free (saving, moving to a GPU, listing parameters). You write only what is specific to your network.
- ▶ `isinstance(c, Player)` is `True` for a Captain: a child object counts as a parent object everywhere. That is why PyTorch accepts your `Net` wherever it wants a `Module`.

Python OOP: **Special Methods and the Dataset Pattern**

Definition

`__len__` tells Python what `len(obj)` returns. `__getitem__` tells Python what `obj[i]` returns. With both, a for loop over your object works too, because for asks for `obj[0]`, `obj[1]`, ... until the end.

You write	Python calls	Must return
<code>len(ds)</code>	<code>ds.__len__()</code>	a whole number
<code>ds[3]</code>	<code>ds.__getitem__(3)</code>	the item at position 3
<code>for item in ds:</code>	<code>ds.__getitem__(0), (1), ...</code>	one item per step
<code>print(ds)</code>	<code>ds.__str__()</code>	a string

Why: a dataset of 150 flowers is “a list of examples”, and the training code in Week 3 wants to ask two things only: how many, and give me number *i*.

```
class FlowerDataset:
    def __init__(self, X, y):          # X: measurements, y: labels
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.y)           # number of examples

    def __getitem__(self, i):
        return self.X[i], self.y[i]   # (measurements, label) of i

ds = FlowerDataset(X, y)              # X, y from Day 02's Lab 2
len(ds)                              # 150
ds[0]                                # (array([5.1, 3.5, 1.4, 0.2]), 'Iris-setosa')
```

Step by step for `ds[0]`: Python sees the brackets, calls `ds.__getitem__(0)`, which reads row 0 of `self.X` and entry 0 of `self.y` and returns them as a pair (a tuple, Day 01).

Today (plain Python)

```
class FlowerDataset:
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return self.X[i], self.y[i]

for x, label in ds:
    ...                                # 150 steps
```

Week 3 (PyTorch)

```
from torch.utils.data import Dataset

class FlowerDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return self.X[i], self.y[i]

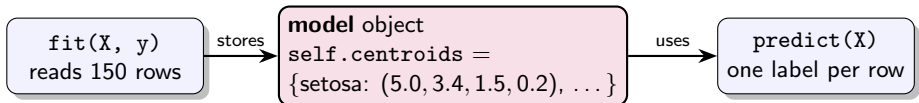
loader = DataLoader(ds, batch_size=32)
```

The only differences: the parent Dataset in brackets, and a DataLoader that calls your two methods to build batches. If `__len__` is missing, the loader fails with `TypeError: object of type 'FlowerDataset' has no len()`.

Python OOP: **A Model as a Class**

The pattern

A **model** is an object with two methods. `fit(X, y)` looks at the training data and **stores what it learned in attributes**. `predict(X)` uses those attributes to answer for new rows. Nothing is passed around by hand between the two calls.



Day 02's Lab 2 computed the centroids in one cell and used them in another, passing a dictionary along. The class keeps that dictionary inside the object, so the whole model is one variable you can pass, copy or save.

```
import numpy as np

class NearestCentroid:
    def __init__(self):
        self.centroids = {}           # empty until fit is called

    def fit(self, X, y):
        for name in set(y):           # each species once
            rows = [x for x, label in zip(X, y) if label == name]
            self.centroids[name] = sum(rows) / len(rows)  # mean
        return self                   # allows NearestCentroid().fit(X, y)
```

- `set(y)` holds the three distinct labels. The comprehension collects one species' rows; `sum(rows)` adds 50 vectors entry by entry (Day 02); dividing by 50 gives the centroid. Everything learned goes into `self.centroids`.

```
def predict(self, X):                                # inside class NearestCentroid
    labels = []
    for x in X:
        best = None
        best_dist = float("inf")
        for name, c in self.centroids.items():
            dist = np.linalg.norm(x - c)             # Day 02: distance
            if dist < best_dist:
                best, best_dist = name, dist
        labels.append(best)
    return labels
```

- ▶ Same loop as Lab 2's predict, now reading `self.centroids` instead of a global dictionary.
- ▶ `predict` does not change the object: it only returns. `fit` changes the object: it only stores. The two kinds of methods from the first section.

Ours

```
model = NearestCentroid()
model.fit(X, y)
preds = model.predict(X)
acc = np.mean(np.array(preds) == y)
print(acc)          # 0.927 (Day 02, Lab
                    2)
print(model.centroids["Iris-setosa"])
# [5.006 3.418 1.464 0.244]
```

Week 2 (scikit-learn)

```
from sklearn.neighbors import
    NearestCentroid
model = NearestCentroid()
model.fit(X, y)
preds = model.predict(X)
acc = np.mean(preds == y)
print(model.centroids_)
# same numbers; the name ends in _
```

Same three lines. Every scikit-learn model (linear regression, decision tree, SVM) is a class with `fit` and `predict`; the learned numbers sit in attributes whose names end with an underscore. When you read `model.coef_` in Week 2, you now know exactly what it is.

Python OOP: Olympiad Practice

Task

The organisers give you this skeleton and a hidden grader that runs `clf = MajorityClassifier(); clf.fit(X_train, y_train); preds = clf.predict(X_test)` and scores the accuracy. Complete the class so that `fit` finds the **most common label** in `y_train` and `predict` returns that label for every row of `X_test`. This is the baseline every Olympiad submission must beat.

```
class MajorityClassifier:
    def __init__(self):
        self.majority = None        # TODO: what should fit store?

    def fit(self, X, y):
        pass                        # TODO

    def predict(self, X):
        pass                       # TODO: one label per row of X
```

Adapted from: the class-skeleton format of the Olimpiada AI (Poland) stage-1 tasks and the fit/predict interface graders use on Kaggle-style contests.

Recognise: the Day 01 counting pattern inside fit, stored in an attribute; predict only reads it. **Check:** `len(preds) == len(X_test)`; on the Titanic table (Day 03) the majority label is 0 and the baseline accuracy is $549/891 = 0.616$.

```
class MajorityClassifier:
    def __init__(self):
        self.majority = None

    def fit(self, X, y):
        counts = {}
        for label in y:
            counts[label] = counts.get(label, 0) + 1
        self.majority = max(counts, key=counts.get) # largest count
        return self

    def predict(self, X):
        return [self.majority] * len(X) # same answer, len(X) times
```

Python OOP: **Check Yourself**

Answer on paper before turning the page.

1. In one sentence: what is the difference between `p.score` and `p.level()`?
2. A class `Teacher(Player)` defines only `__init__(self, name, subject)` with `super().__init__(name)` and `self.subject = subject`. Does `Teacher("Lina", "math").add_points(5)` work? What does `print(Teacher("Lina", "math"))` show?
3. What does this print, and why?

```
class Bag:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)
    def __getitem__(self, i):
        return self.items[i] * 2

b = Bag([1, 2, 3])
print(len(b), b[1], [x for x in b])
```

1. An **attribute** is data stored in the object (`self.score`); a **method** is a function that belongs to the object and takes `self`. `p.score` reads the stored number, no call; `p.level()` runs the method and returns a computed number.
2. Yes: `add_points` is inherited from `Player`, and `super().__init__(name)` created `self.score = 0` first, so the score becomes 5. `print` shows `Player Lina: 0 points`, because `Teacher` did not override `__str__`, so the parent's version runs.
3. `3 4 [2, 4, 6].len(b)` calls `__len__`, which is 3. `b[1]` calls `__getitem__(1)`: item 2, doubled, is 4. The comprehension loops with `__getitem__(0)`, (1), (2) and stops at index 3 (an `IndexError` inside `__getitem__` ends a for loop), giving the doubled items.

Python OOP: **Limitations & What Comes Next**

The mistakes to watch for

- ▶ **Forgetting** `self` in a method definition: `TypeError: level() takes 0 positional arguments but 1 was given`. Python passed the object; the method had no parameter for it.
- ▶ **Forgetting** `super().__init__()` in a child: the parent's attributes are never created, and the first `self.score` fails with `AttributeError`.
- ▶ **Calling** `predict` **before** `fit`: the attributes are still empty. `scikit-learn` raises `NotFittedError` for exactly this reason.
- ▶ **Storing the answer in a global variable** instead of `self`: it works once, then breaks when you have two models.
- ▶ **Skipped on purpose**: class attributes shared by all objects, private names, properties, and multiple inheritance. You will not need them in this course.
- ▶ **Tomorrow, Day 05 (self-study)**: NumPy in depth, so the loops inside `fit` and `predict` become `X[y == name].mean(axis=0)` and one broadcasted distance.
- ▶ **Week 2**: you use the `scikit-learn` twins of today's class; **Week 3**: you write `Dataset` and `nn.Module` children for real.

Python OOP: **Summary**

- ▶ **Attributes** hold the object's data; **methods** act on it. A method either changes `self` or returns an answer; read its last line.
- ▶ `__str__` makes `print(obj)` useful. Double-underscore methods are called by Python at fixed moments, never by you.
- ▶ **Inheritance** builds a child on a parent: name the parent in brackets, call `super().__init__()` first, add or override what differs. `nn.Module` is the parent of every PyTorch model.
- ▶ `__len__` and `__getitem__` make your object work with `len()`, `[]` and `for`: the **Dataset pattern** of Week 3.
- ▶ A **model class** stores what `fit` learned in attributes and reads them in `predict`: the shape of every scikit-learn model.
- ▶ The majority-class baseline is a ten-line class in this shape, and the first thing to beat in any competition.

Python OOP: **References & Resources**

- [1] Python Software Foundation (2026). *The Python Tutorial*, chapter 9: Classes.
<https://docs.python.org/3/tutorial/classes.html>
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR* 12. The fit/predict interface and NearestCentroid: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestCentroid.html>
- [3] PyTorch (2026). *Datasets & DataLoaders* tutorial (the `__len__`/`__getitem__` pattern).
https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
- [4] Olimpiada AI (Poland), stage-1 task notebooks (function and class skeletons with local tests), which the practice task imitates.
https://github.com/OlimpiadaAI/II-OlimpiadaAI/tree/main/1_etap
- [5] Altwaim, S. (2026). *Day 1: Math & Code Foundations*, KAUST Academy & Mawhiba IOAI 2026 training. The Dataset/ImageDataset inheritance example is adapted from it.